

InclusiVenue

Plateforme de réservation d’événements culturels et associatifs accessibles

Mohamed Haouari · ESPRIT — Projet intégré FullstackJS · 2026-2027

Le problème : savoir si l'on pourra venir

Aujourd'hui

- Langue des signes, boucle magnétique, accès fauteuil : l'information est absente ou dispersée.
- Il faut appeler l'organisateur avant de réserver.
- Les associations n'ont pas d'outil simple pour gérer jauges et besoins.

InclusiVenue

- 12 services d'accessibilité décrits, filtrables, vérifiés en modération.
- Réservation garantie, besoins transmis à l'organisateur.
- Une interface elle-même accessible (WCAG 2.1 AA visé).

Public visé : personnes en situation de handicap visuel, moteur, auditif ou cognitif, leurs accompagnants, et les associations culturelles.

Quatre rôles, contrôlés côté serveur



Visiteur

Parcourir, rechercher, filtrer, consulter la carte



Participant

Réserver, annuler, iCal, notifications, préférences



Organisateur

Créer, publier, gérer jauges, pointer, statistiques

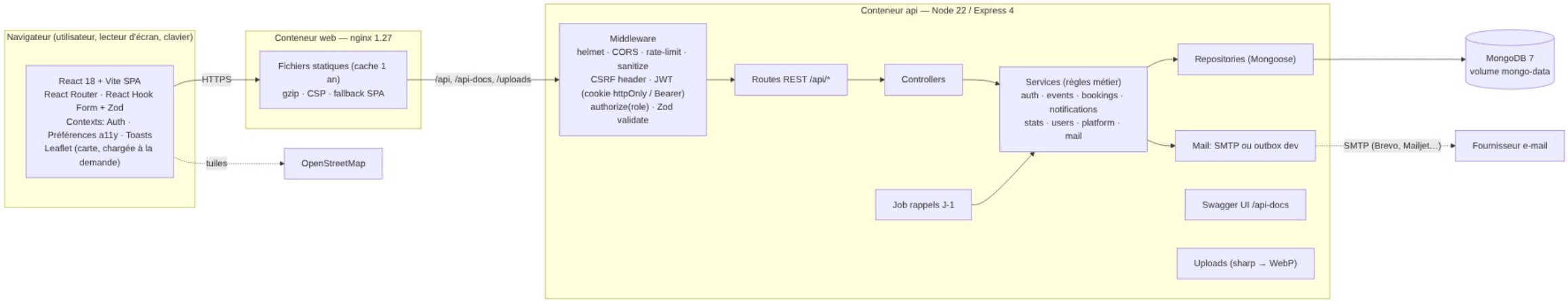


Administrateur

Modérer, gérer comptes, catégories, paramètres

Chaque route protégée applique authentification + rôle, et les services revérifient la propriété : masquer un bouton n'est jamais la sécurité.

Architecture en couches



Frontend	React 18 · Vite · React Router · React Hook Form + Zod · CSS Modules
Backend	Node 22 · Express · Zod · JWT cookie httpOnly · bcrypt · helmet · rate limit
Données	MongoDB 7 · Mongoose · index texte, 2dsphere, unique partiel
DevOps	Docker Compose · nginx · GitHub Actions · Playwright · k6

Zéro surréservation, même sous la concurrence

1

Vérifier

Publié, non commencé, plafond de places, pas de réservation active

2

Décrémenter

findOneAndUpdate conditionnel : availableSeats ≥ n → atomique

3

Enregistrer

Index unique (user, event, confirmée) ; conflit → places rendues

4

Notifier

Participant + organisateur, e-mail selon préférence

Vérifié : 20 utilisateurs simultanés pour la dernière place → exactement 1 réservation (test unitaire exécuté). Le même scénario sur un vrai MongoDB (25 requêtes, 3 places) est écrit en test d'intégration, à exécuter.

Accessible par conception, pas après coup

Clavier

Lien d'évitement, focus visible, focus déplacé à chaque page, dialogues piégés + Échap

Affichage

Texte 100–200 %, contraste élevé, animations réduites, lecture facilitée

Formulaires

Labels, aides et erreurs liés, récapitulatif focalisé

Profils

Mode lecteur d'écran, mode sourd/malentendant, besoins par défaut

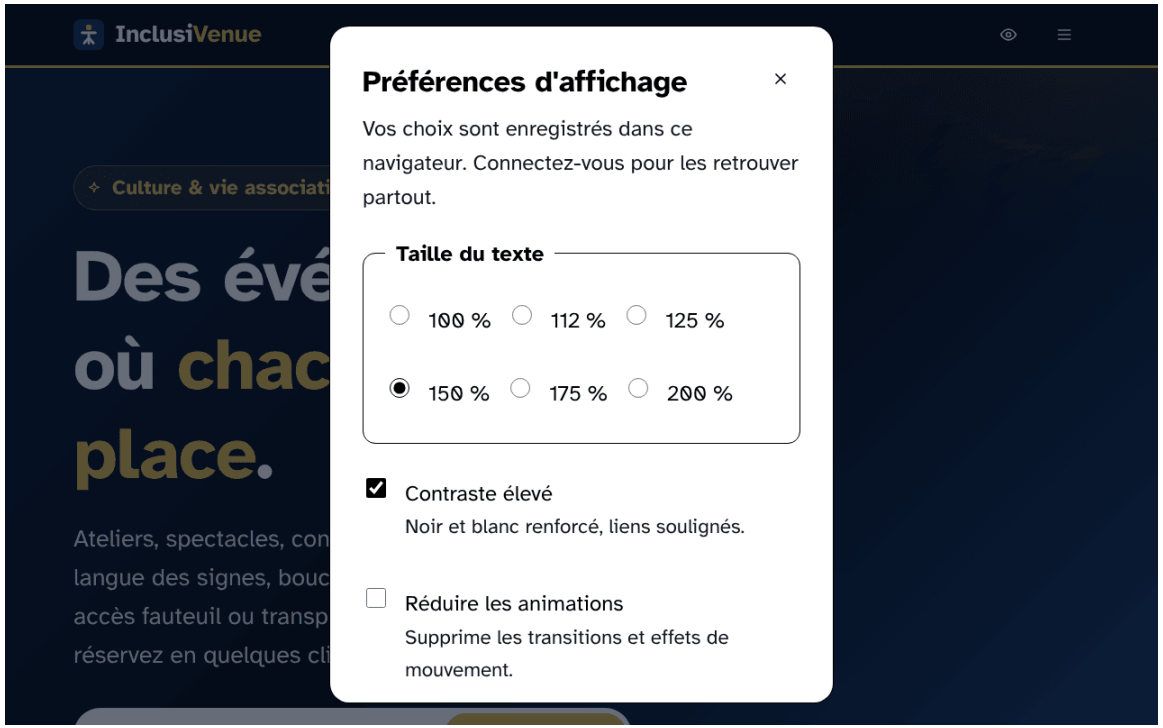
Annonces

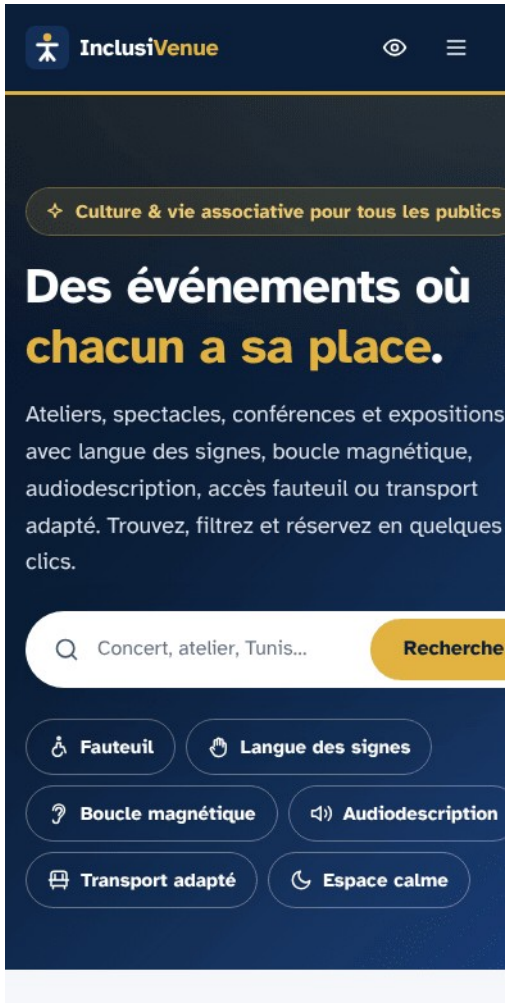
Notifications et résultats dans des régions aria-live

Contrastes

Tous les textes ≥ 4,5:1 — 2 598 nœuds mesurés, 0 sous le seuil

Le même écran, adapté à chacun

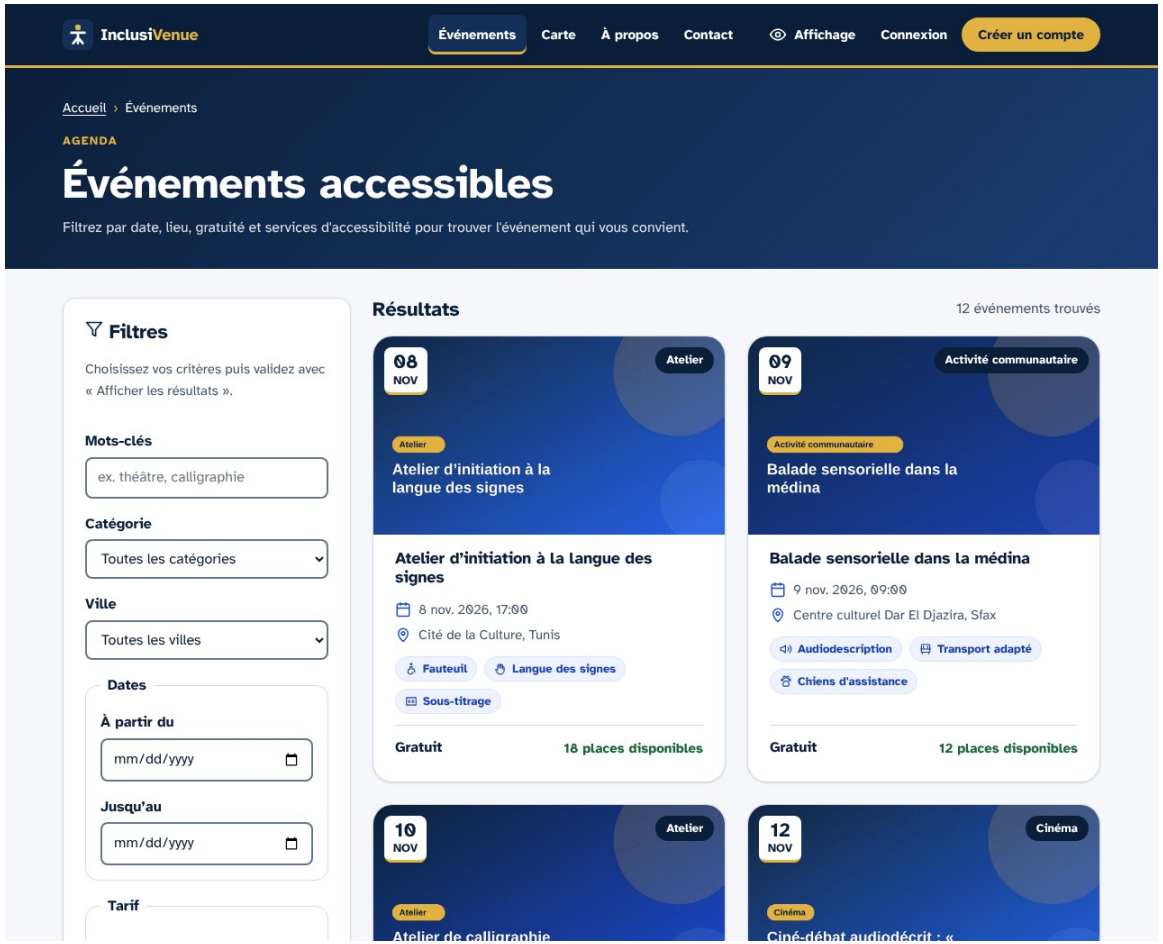


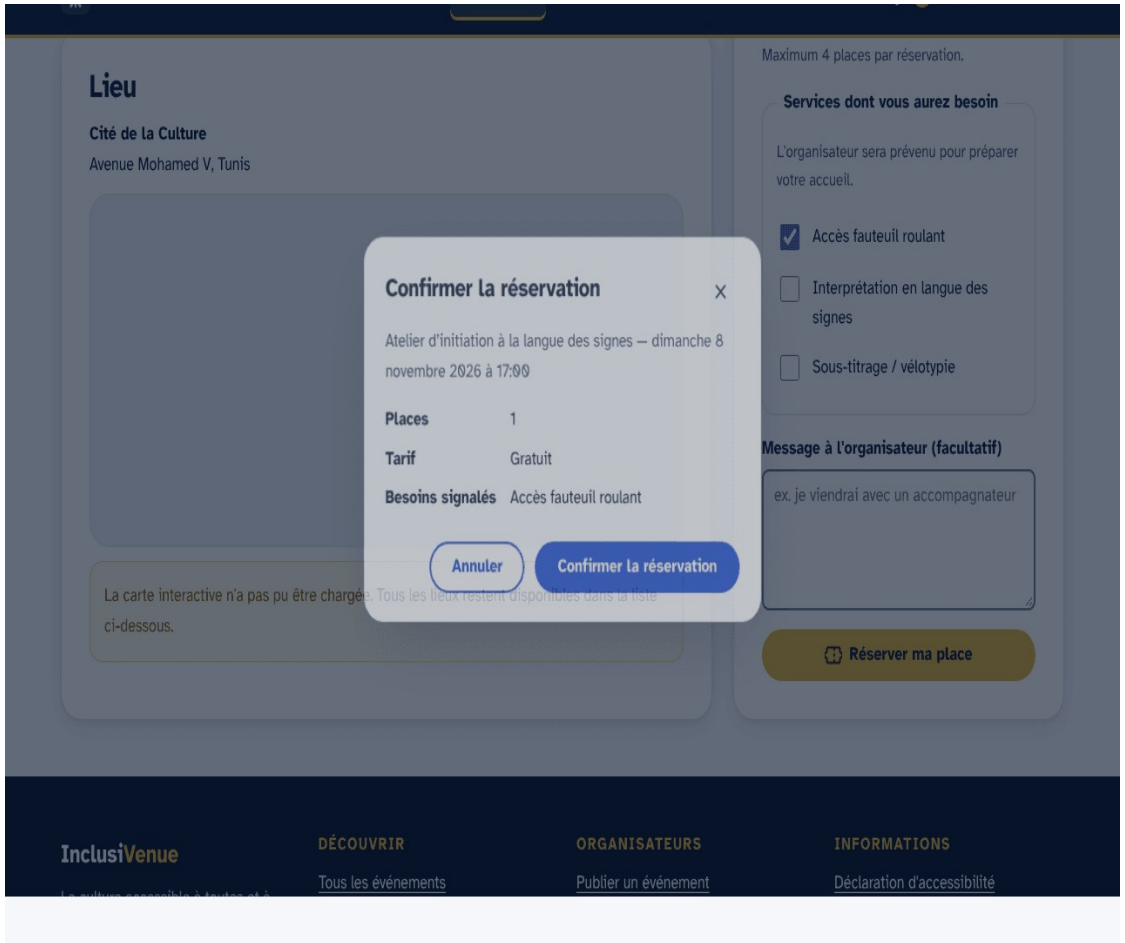




Préférences d’affichage (contraste, 150 %) · mobile 390 px · récapitulatif d’erreurs accessible

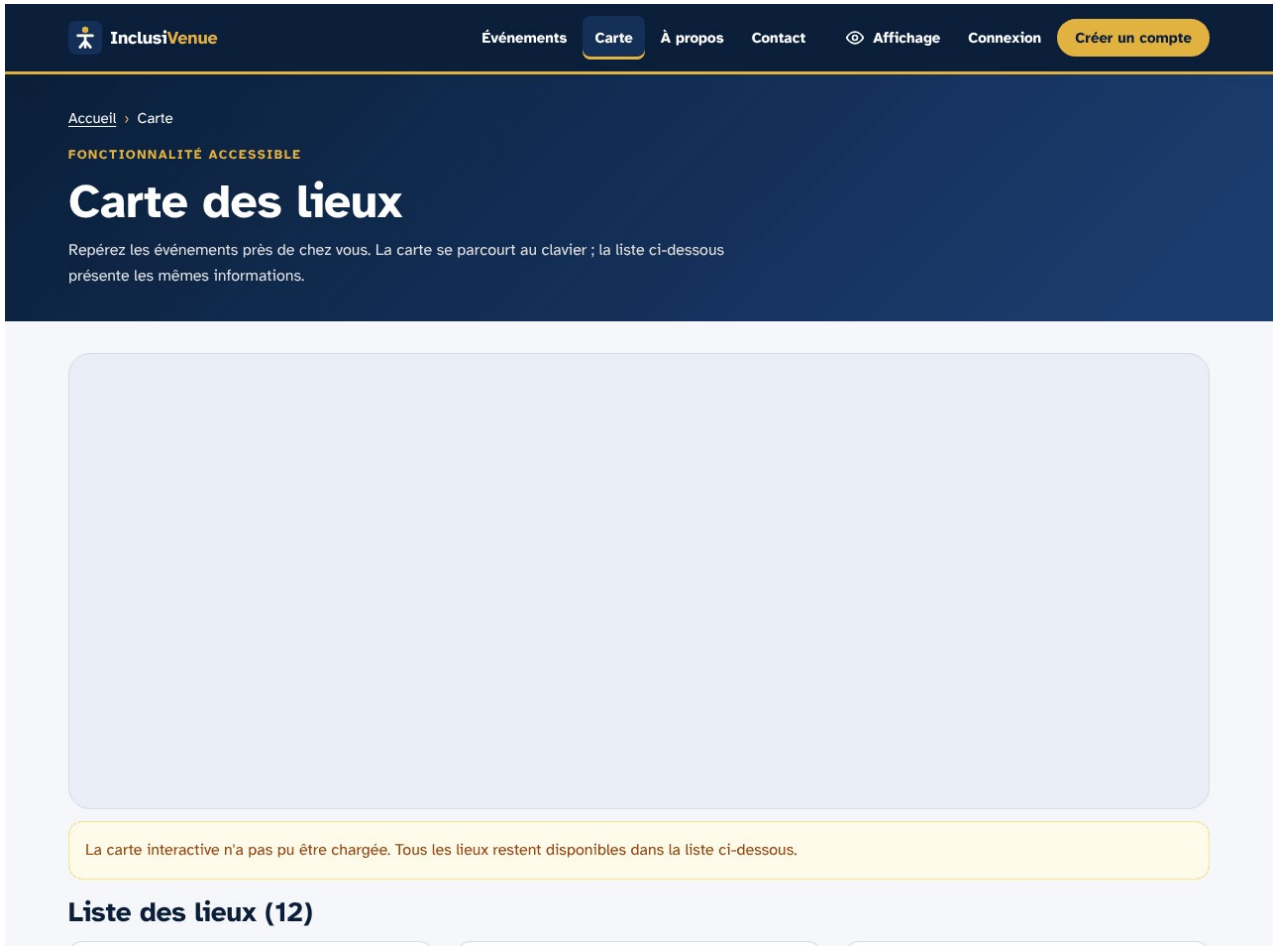
Parcours participant





Filtrer par service → fiche → choisir places et besoins → confirmer → notification + iCal

Différenciant : carte accessible, iCal, lecture vocale



Carte Leaflet au clavier

Marqueurs focusables, Entrée ouvre la fiche ; liste textuelle équivalente toujours affichée

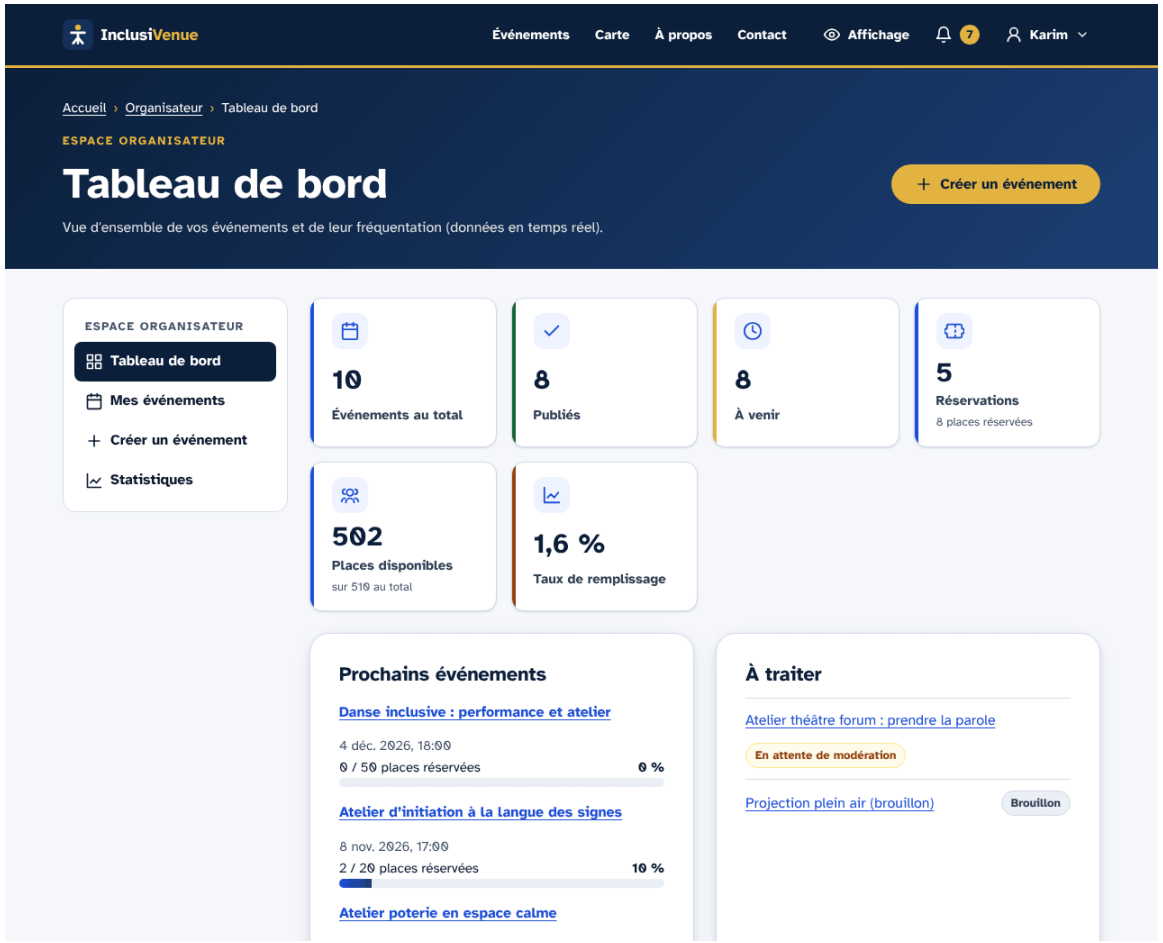
Export iCal

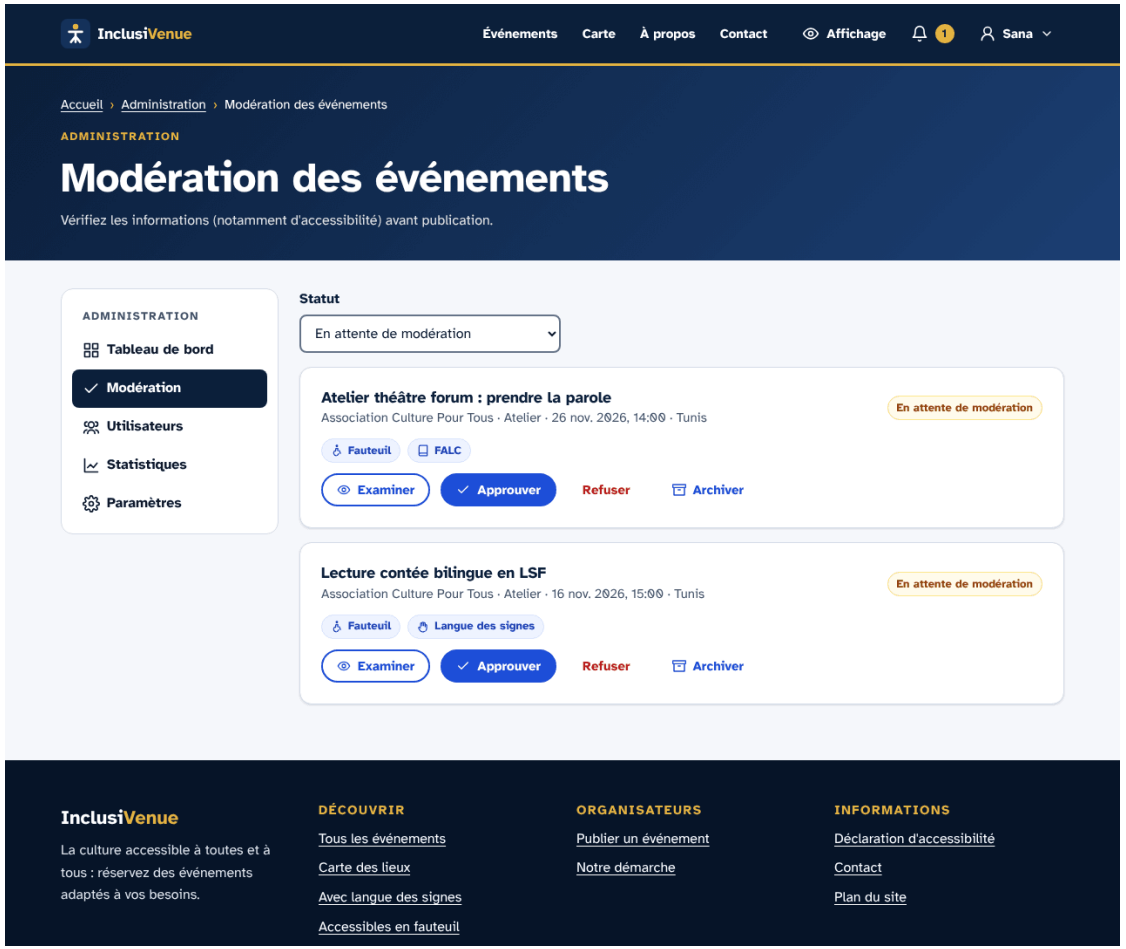
Toutes mes réservations ou une seule, avec rappel 24 h

Écouter la description

Synthèse vocale du navigateur, aucune donnée envoyée

Organiser et modérer, sur des données réelles





Statistiques calculées par agrégations MongoDB · modération avec motif · pointage et export CSV

Tests : ce qui est prouvé, ce qui reste à exécuter

99/99

tests unitaires backend réussis

91,6 %

lignes couvertes (couche métier)

17/17

tests E2E réussis (banc)

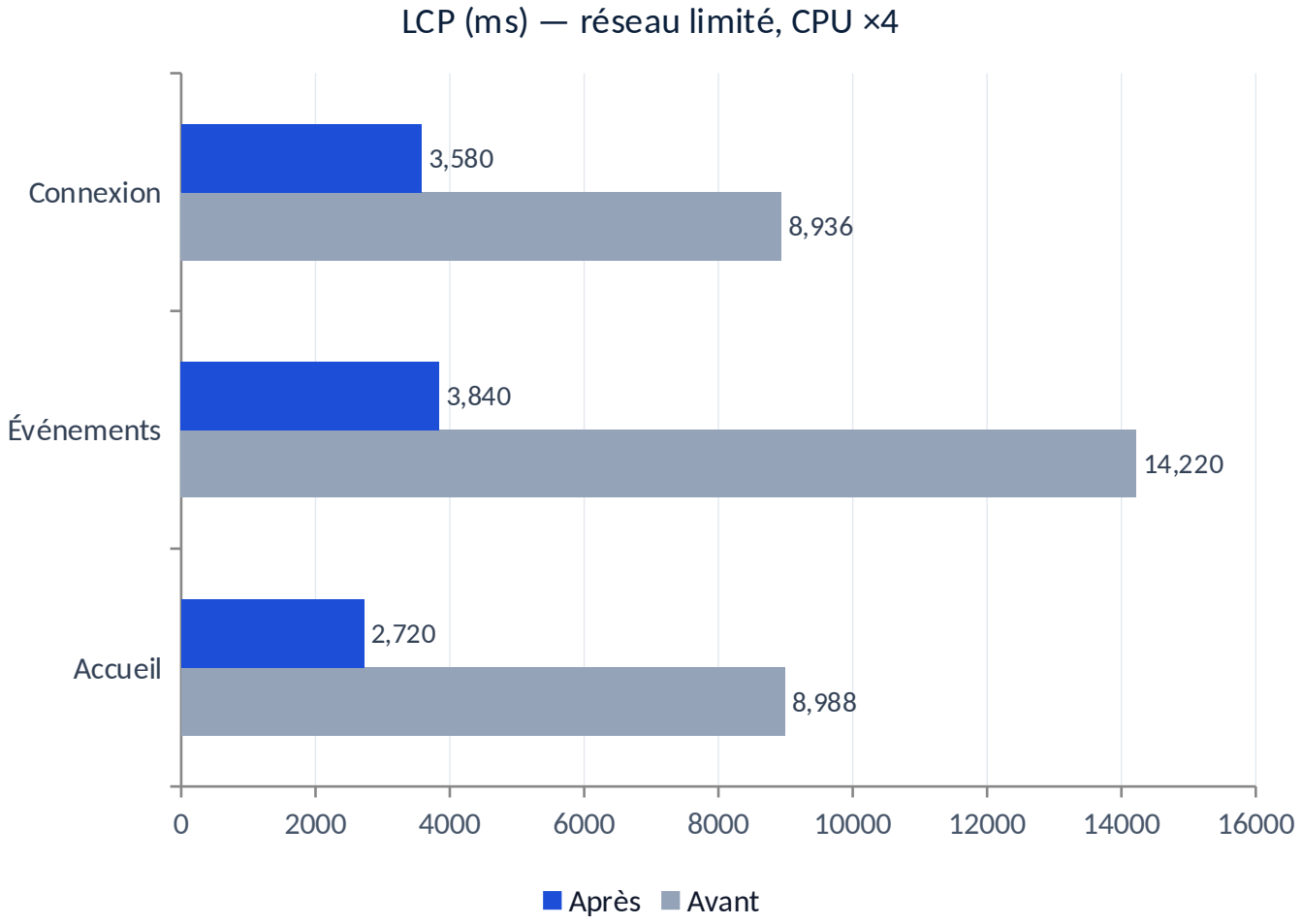
0

anomalie sur 30 écrans contrôlés

À exécuter sur une machine avec npm et Docker

- 25 tests d'intégration (API + MongoDB), 10 audits axe WCAG 2.1 AA, Lighthouse, WAVE, k6, docker compose up
- L'environnement de développement n'avait pas accès au registre npm : aucun résultat n'est présenté pour ces suites.

Performance : avant / après optimisation



−69 %

JavaScript initial (284 → 87 Ko gzip)

−92 %

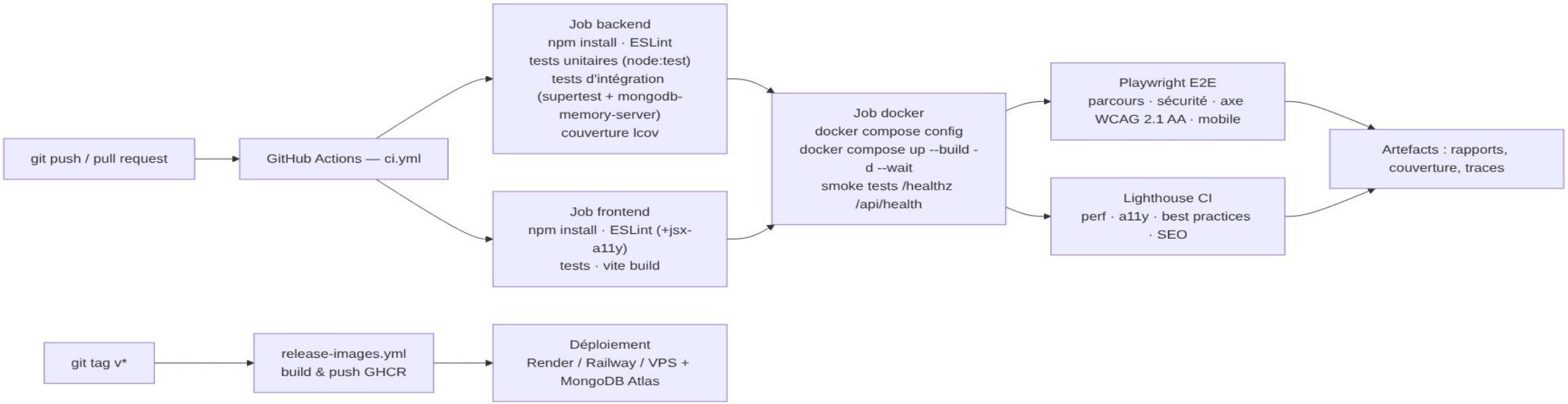
images de la liste (WebP)

0,376 → 0

CLS de la liste

Mesuré sur le banc local (Playwright). Lighthouse et k6 à mesurer sur la stack réelle.

DevOps : de git push au déploiement



`docker compose up --build -d`
MongoDB, API et nginx avec health checks et volumes persistants ; CI : lint, tests, build, E2E + axe, Lighthouse ; images publiées sur GHCR à chaque tag.

Prochaines étapes

- Exécuter intégration, axe, Lighthouse, k6 et Docker ; compléter les rapports
- Déployer (Render + MongoDB Atlas) et brancher un SMTP réel
- Tests avec NVDA / VoiceOver et des utilisateurs concernés

Merci — questions ?